

第一階段 PoC 成果與下一步決策

分散式資料庫架構 × 跨區域驗證 × 決策框架

2026-06-26 | DBA

Outline

- 專案迄今歷程
- 第一階段 PoC 測試數據彙整
- 第二階段 跨區域 / 跨專線執行進度
- 決策框架說明
- 後續推進

專案歷程 ① 研究定義到跨家框架

先定義問題與比較口徑，再建立可重複執行的基礎設施與三資料庫共同工具鏈

時間	階段	重大節點	狀態
2026-03-30～04-10	前期研究	定義分散式 SQL、跨區同鍵寫入、Follower Read、HA/DR 評估面向	✅ 完成
2026-04-21～04-27	IaC 與第一版測試鏈	建立多測項部署及獨立壓測流程 (Flow Chain) A -> B -> C -> D -> E	✅ 完成
2026-04-28～05-05	YugabyteDB 首輪除錯	處理 BenchmarkSQL、bulk load、RF / schema packing 與 HAProxy issue	✅ 完成
2026-05-06～05-14	三資料庫對標成形	納入 TiDB、CockroachDB、YugabyteDB，統一結果結構與 go-tpc 工具鏈	✅ 完成

此階段完成的是「可比較的工程框架」，早期測試數字不直接納入 v4.7 正式結果。

專案歷程 ② 基準、三節點與治理

將測試從可執行提升為可重現、可追溯、可拆解成本來源

時間	階段	重大節點	狀態
2026-05-18～05-21	v4.7 baseline 重構	建立 PoC-DESIGN SSOT、detached suite、gate、marker、summary 與單節點三隔離級對標	✅ 完成
2026-05-22～06-02	三節點 controlled experiment	完成 shard × replica × HAProxy 拓樸、12-cell dry-run 與資料庫數據初步彙整	✅ 完成
2026-05-20～06-04	文件與數據治理	建立模板、Agents 協作規範、artifact-first 實測數據校驗與 pipeline-log 對齊	✅ 完成
2026-06-06～06-07	Phase isolation	分離 S-BASE、S-K8S、T-THRD、X-CROSS，建立配置宣告、基礎指標確認及指標配置	✅ 完成

正式數字必須能追回測試條件、時間戳、結果檔案與完成標記；缺少來源資料 (測試階段) 時不予納入參考。

專案歷程 ③ Kubernetes 到跨區驗證

已完成 Kubernetes 對照與跨區技術路徑；正式跨區效能仍受實測結果約束

時間	階段	重大節點	狀態
2026-06-08～06-14	Kubernetes	由單 cell dry-run 擴充至三資料庫 × limit/unlimit 六組正式 suite	✅ 6/6 完成
2026-06-08～06-17	跨區設計與前置開發	建立 IDC + GCP 六節點部署、placement、WAN、chaos、failover 與 pre-flight 規格	● 框架完成 部分僅 dry-run
2026-06-18～06-19	IDC↔GCP 實際驗證	修正 IaC、防火牆與 YugabyteDB placement, 各 DB 完成六節點前期測試	✅ smoke 完成
2026-06-21～06-22	Determinism 收斂	W=4 重跑浮動過大, 改採同 cluster、變異量/變異參數 與 W=128 baseline 對標	● 進行中

決策界線：可確認六節點跨區交易路徑可行；正式跨家排序須等 W=128、R2～R5 中位數 / 變異係數與回復流程驗收結論。

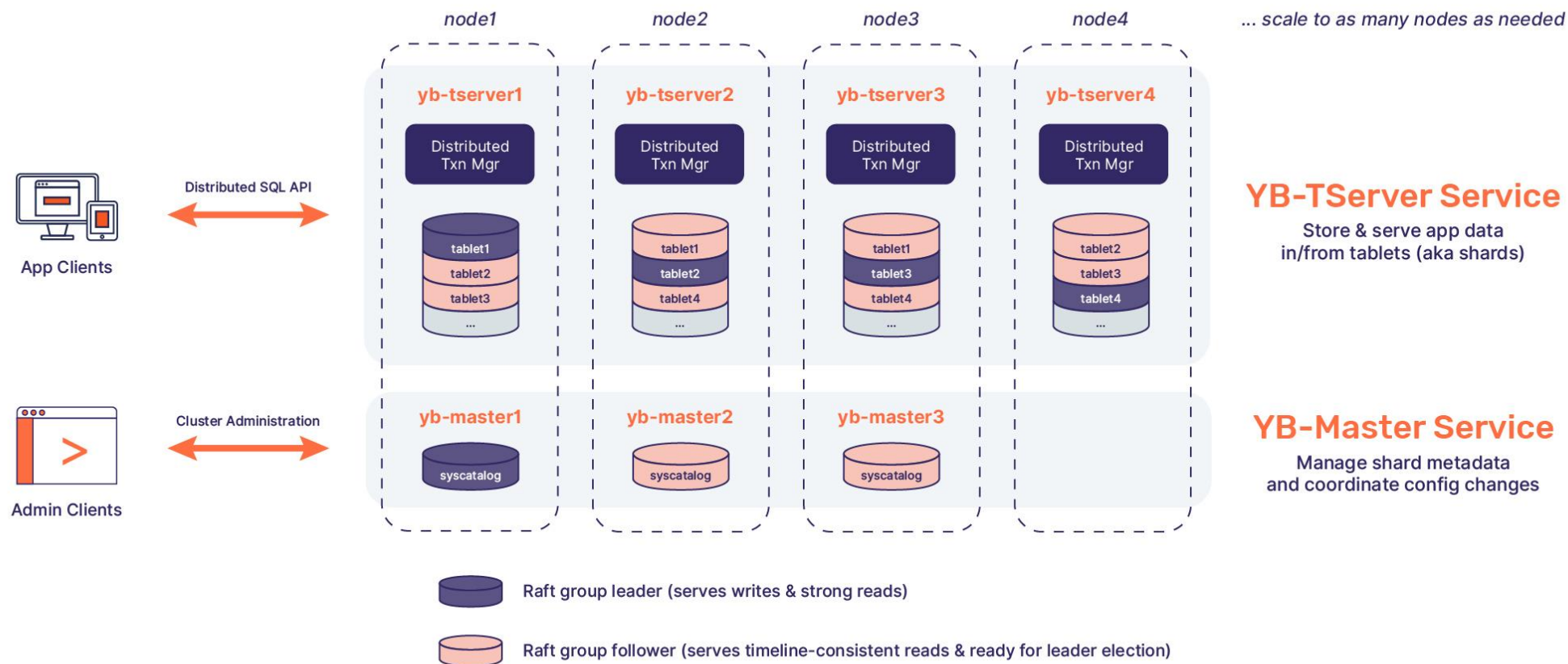
三家資料庫導入定位

資料庫	第一階段觀察	導入定位
TiDB	VM 與 K8s 吞吐表現較佳；K8s retention 較高	方便優先進入應用情境對接 (MySQL Compatibility)
CockroachDB	一致性需求較強（SSI 預設）；SI / SSI 模式下 retry 與效能成本明顯	保留觀察：應用層需評估 Error Handling 容忍度與交易模式
YugabyteDB	VM + HAProxy 表現為佳；K8s 結果目前不宜直接作為導入結論	保留觀察：K8s 部署仍需調校與驗證，VM 路徑可進入評估

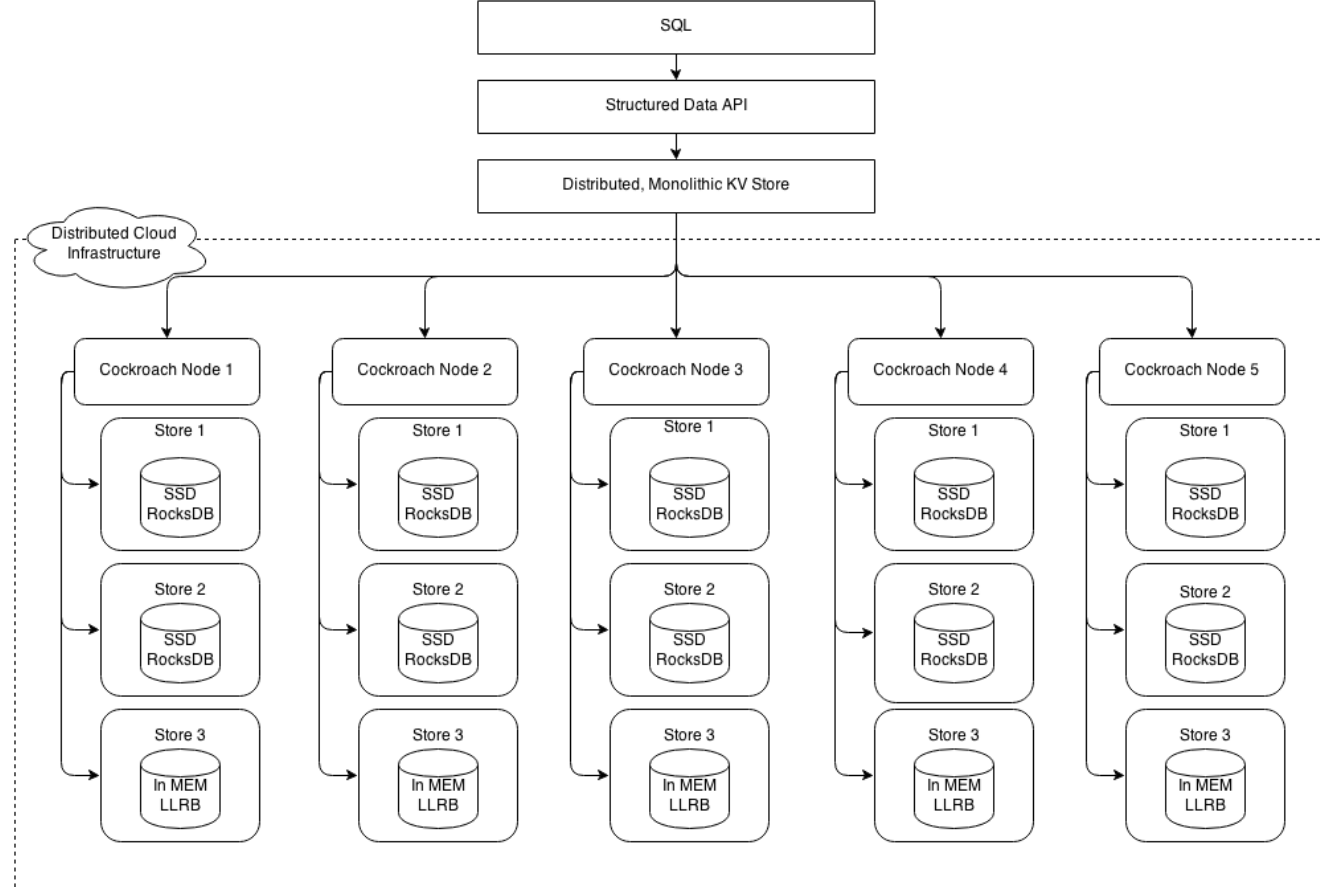
同 IDC / 同硬體 基準 — vm-3node-haproxy-3s3r-rc, t=128 mean tpmC

TiDB	YugabyteDB	CockroachDB
≈ 26,900	≈ 15,600	≈ 15,000

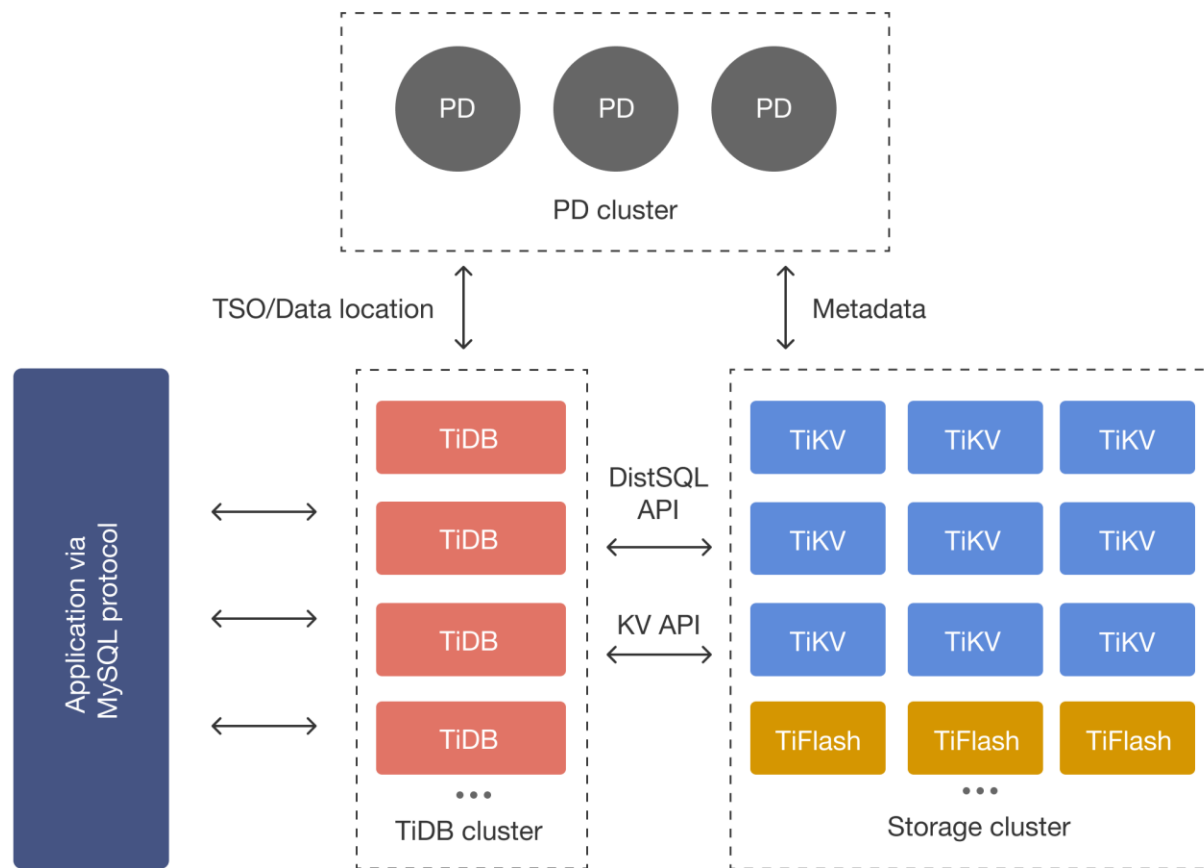
本組數字為 controlled experiment 基準（VM 現況資源 × 3 node 拓樸）；不直接代表各家「拿出來就跑」的生產表現；不具任何採購驗收指標用。
商業實體 / 授權 / 採購層面議題需另案審查。
Ref: <https://docs.yugabyte.com/stable/benchmark/tpcc/>



- **YSQL** : PostgreSQL-compatible SQL API , 負責接收 SQL、處理 query planning 與 transaction request 。
- **YCQL** : Cassandra-compatible API , 本輪 TPC-C-derived OLTP 測試不走此路徑。
- **DocDB** : YugabyteDB 的 distributed document store , 負責資料儲存、MVCC、tablet 管理與 Raft 複寫。
- **YB-TServer** : 資料服務節點, 承載 YSQL/YCQL request path 與 DocDB tablet ; SQL 與儲存同在 tserver 內, 和 TiDB 的 SQL / storage 分離不同。
- **YB-Master** : 叢集 metadata 管理元件, 負責 tablet placement、schema metadata 與 cluster coordination 。
- **架構重點** : tserver 一體式架構 ; 加節點時 SQL 接收能力與資料 tablet capacity 一起增加, 實際效能同時受 tablet 分布、複本數與 transaction coordination 影響。



- **SQL layer**：每個 CockroachDB 節點都能接 SQL request、產生 query plan、處理 transaction coordination。
- **Transactional KV / Distribution**：SQL 會轉成 key-value operation，由 distribution layer 找到資料所在 range 並路由到正確節點。
- **Replication / Raft**：資料以 range 為單位複寫，透過 Raft 維持一致性；replica 數會直接影響寫入 quorum 與 commit latency。
- **Storage**：每個節點都同時保存資料並處理查詢，沒有獨立 SQL node / storage node 分層。
- **架構重點**：對稱式架構；任一節點都同時具備 SQL、transaction、distribution、replication 與 storage 能力，HAProxy 可把連線分散到多個完整節點。



- **TiDB server**：SQL 接收層，負責解析 SQL、產生執行計畫、處理 transaction coordination；本身不保存主要資料。
- **TiKV**：分散式 row store，保存 OLTP 資料；資料以 Region 切分並透過 Raft 複寫，影響寫入延遲與多副本成本。
- **PD (Placement Driver)**：叢集 metadata 與排程控制中心，負責 timestamp oracle、Region placement 與負載調度。
- **TiFlash**：columnar replica，主要用於 HTAP / analytical query；本輪 TPC-C-derived OLTP 測試不以 TiFlash 為主要路徑。
- **架構重點**：SQL 層與儲存層分離；增加 TiDB server 可擴充 SQL 接收與執行能力，增加 TiKV 則擴充資料儲存與 Raft 複寫能力。

分散式 DB 跨區後，延遲稅集中在 commit 路徑上「必跨 WAN 的節點」且設計機制不同

- TiDB 走中央 PD TSO（每 txn 取 timestamp 1 RTT），
 - CockroachDB / YugabyteDB 用 HLC 本地時鐘（無 TSO RTT），
 - 三家共通的是 Raft quorum ACK — leader 與 voter 若跨區即 +1 RTT。
-
- YSQL postgres backend 與 DocDB tserver 是兩層進程邊界，請求需經 RPC / 序列化 / 反序列化；若再疊加跨 Pod / 跨節點路由，延遲與 CPU 開銷會進一步放大。
-
- 跨區後維持強一致 = 把跨 WAN 的 RTT 算進 commit latency = 無法用最終一致性換寫入速度
 - 強一致與跨區效能無法同時最佳化 — 必須選 workload 模式（A-S / A-A-RO / A-A）。

分散式 DB 把 table 切成 shard 分散到各節點，「row 屬哪個 shard」有兩種演算法：

- Range sharding：依 key 區間切（id 1-1000 → shard-1）
 - 範圍掃描快、有 locality 但 sequential write（auto-increment / timestamp）會集中熱點。
- Hash sharding：對 key 做 hash 取模分配。寫入分散無熱點
 - 但範圍掃描需 fan-out 到所有 shard，跨 shard JOIN 成本高
- TPC-C 的 NEW_ORDER / PAYMENT 高度依賴 warehouse 相關 row 的 locality
- Hash sharding 在 hot warehouse 場景反而能避開單 shard 飽和

TPC-C 同硬體 ranking：TiDB > CRDB ≈ YBDB 部分受此影響
（YBDB 預設 hash 在 warehouse-centric workload 下 locality 較差）

觀察	解釋
TiDB tpmC ≈ 26,900（最高）	Range + warehouse-aware split + leader balance 對 OLTP locality 最友善
CockroachDB tpmC ≈ 15,000	Range 但 SSI 預設 + retry 成本拉低吞吐
YugabyteDB tpmC ≈ 15,600（VM）	Hash 預設打散，無熱點但 cross-tablet txn 比例高，吞吐受限於 commit fanout

VM 與 Kubernetes 效能差異 (pre-view)

K8s 化對 TiDB 與 CockroachDB 屬可接受範圍；YugabyteDB 在 K8s 下退化顯著，列為調校與驗證項

資料庫	S-BASE (VM) tpmC	S-K8S unlimit tpmC	S-K8S limit tpmC	unlimit / VM	limit / VM	p99 unlimit Δ	p99 limit Δ
TiDB v8.5.2	26,947	23,442.9	15,751.9	87.0 %	58.5 %	+17 %	+111 %
CockroachDB v26.2	15,033	12,196.7	6,493.5	81.1 %	43.2 %	+27 %	+192 %
YugabyteDB 2025.2	15,632	2,997.6	1,604.5	19.2 % ⚠	10.3 % ⚠	+669 %	+1556 %

觀察解讀

- TiDB / CockroachDB：K8s unlimit 保留率 81-87 %，屬可接受範圍；limit 情境大幅衰退反映資源壓縮
- YugabyteDB：K8s unlimit 僅 19.2 %、p99 +669 %。成因推論：YSQL + DocDB 雙 process 在 K8s pod IPC + CPU contention 放大。**成因未定位，列後續調校項**
- S-BASE ↔ S-K8S 可直接比較（同 workload W=128 t=128）

跨區域 / 跨專線進度

✓ 已完成

- 跨機房技術議題定案（GCP 5 VM 拓樸、Placement P-A / P-B (註)、Chaos Engineering & Failover Scenarios design）
- IaC、playbook、suite scripts、chrony drift gate（IDC & GCP 時間偏移: 0.017 ms）
- 06-18～06-19：修正 IaC / 防火牆 / YugabyteDB Placement，三家 DB / 六節點前期測試驗證
- 06-21～06-22：Determinism v2 — 同 cluster R1-R5、scheduler / balancer freeze testing.

● 待執行

- 調度 scheduler (開/關)、round-only runner、Warmup、Placement (replica / lease 位置)、測試結果彙整
- 回到 W=128 正式基準量測
- Failover 與 Chaos Engineering 實際驗證

為什麼需要 P-A / P-B 兩種 Placement ？

跨區 6-node cluster（3 IDC + 3 GCP）下，Voter 與 Leader 在哪一區 決定 raft commit 是否走 WAN。
把 placement 拆成兩種對照設定，量化 IDC-only baseline → 跨區寫入的 tpmC 衰減：

- P-A：情境「正常營運」— IDC 為主，GCP 為備
- P-B：情境「退化 / Failover 後」— leader 散在兩區，WAN 進入 critical path
- 不同 Placement 對應不同 workload 模式（A-S / A-A-RO / A-A），也對應 chaos 測試的不同 baseline。

X-CROSS 初步結果 (06-22~06-23)

W=4 framework 驗證；不與 S-BASE/S-K8S 直接對比 · 同 cluster 5 rounds、W=4、16 threads、每 round 5 min、controller = .31 (IDC client)

資料庫	R1	R2	R3	R4	R5	中位數 (有效輪)	CV
TiDB v8.5.2	9,525.5	9,553.2	9,786.9	9,393.2	9,530.8	9,566.0 (R2-R5)	1.67 %
CockroachDB v26.2	8,409.5	8,055.3	7,902.5	7,720.9	7,472.3	7,787.8 (R2-R5)	3.23 %
YugabyteDB 2025.2	102.0	226.9	6,424.2	6,259.3	6,206.2	6,296.6 (R3-R5)	1.82 %

已確認結論

- 06-21 觀察到的 $\pm 526\%$ / $\pm 50\%$ 變異主因為「每輪重新部署」造成 placement / cache / scheduler 狀態異動，與 W=4 contention 為兩個獨立來源
- 同一 cluster 連跑時，三家 W=4 R2-R5（或 YBDB R3-R5）的 $CV \leq 5\%$ ，重現性已建立
- YBDB Idle=0 解法：gate 改用 get_load_move_completion=100%；timed run 前 set_load_balancer_enabled=0
- CRDB lease gate SQL 改用 SHOW RANGES FROM DATABASE tpcc WITH TABLES, DETAILS（v26.2 相容）

決策框架建議

不是選邊站，而是先界定 Application 條件、再排序候選；僅提供做決策框架參考依據用

分類	內容	處置
短期候選	TiDB	優先進入應用情境對接，無門檻
保留觀察	CockroachDB	依一致性需求、Error Handling 容忍度、維運成本評估
保留觀察	YugabyteDB	VM 路徑可評估；K8s 路徑需先完成部署層級調校與驗證
暫不作結論	跨區域場景、K8s 退至未來決定項	待下一階段 W=128 測試數據與調校產出再回頭評估

Application 需要確認的議題

- **交易一致性需求**：是否需要 SERIALIZABLE / SSI？是否可接受 READ COMMITTED & 產品設計架構轉 CAP 的複雜度及可行性？
- **可接受延遲**：在尖峰 t=128 等級負載下，p99 是否需要 500ms / 1s / 2s 以內？
- **Error Handling / timeout 行為**：應用層是否能接受 SI / SSI 模式下的 retry 機制？
- **RTO / RPO**：跨區域 failover 是否需要 < 30s RTO？是否容許資料 lag？
- **連線層 / 交易模式調整**：是否能配合 HAProxy / PgBouncer / 連線池與短交易模式？

後續推進階段

Y26/07 (Phase: Cross Region)	Y26/08 (Phase: Cross Region)	Y26/09
Cross Region W=4 框架已完成初步流程驗收， 啟動正式 W=128 基準量測。	- A-S / A-A-RO / A-A 三個 workload profile. - Placement P-A/P-B × 各資料庫，依檢驗指標執行	相關文件產出 & 後勤對接狀況說明
	- Failover 與 Chaos Engineering 測試 - 跨區域測試第二階段報告產出	